Minor Project1

Traffic Light Controller

Submitted By:

Name: Reva Pradip Deshpande

Abstract

Traffic signal systems play a crucial role in regulating vehicle movement and ensuring road safety. This project presents the design and simulation of a Traffic Light Controller using Verilog Hardware Description Language (HDL). The controller is implemented as a Finite State Machine (FSM) consisting of three states: Green, Yellow, and Red. A timer controls the duration of each signal, allowing the controller to transition automatically between states in a predefined sequence. The design follows the three-process FSM methodology, separating the state register, next-state logic, and output logic for improved readability and modularity. The complete design was verified using a Verilog testbench in EDA Playground. Simulation waveforms confirmed the correct sequence of state transitions and proper operation of the controller. This project demonstrates the practical implementation of sequential digital circuits and provides a strong foundation for designing more advanced traffic management systems.

1. Introduction

Traffic control systems are essential for maintaining road safety and ensuring the smooth movement of vehicles. Modern traffic signals use programmable digital controllers to manage traffic flow efficiently. Finite State Machines (FSMs) provide a systematic approach for implementing such controllers. In this project, a Traffic Light Controller is designed using Verilog HDL and simulated using EDA Playground. The controller changes the traffic signals according to predefined timing intervals while maintaining synchronous operation with the system clock.

2. Objectives

- To understand the concept of Finite State Machines.
- To design a Traffic Light Controller using Verilog HDL.
- To implement the three-process FSM methodology.
- To verify the controller through simulation.
- To analyze the timing behavior using waveform results.

3. Theory

A Finite State Machine (FSM) is a sequential digital circuit that transitions between a finite number of states based on the current state and input conditions. In this project, the FSM consists of three states:

- **GREEN:** Vehicles are allowed to move.
- **YELLOW:** Warning state before changing the signal.
- **RED:** Vehicles must stop.

The controller changes from Green to Yellow, Yellow to Red, and Red back to Green after predefined timer values. The design follows the Moore FSM model, where the outputs depend only on the current state.

4. System Design

Inputs

- Clock (`clk`)
- Reset (`reset`)

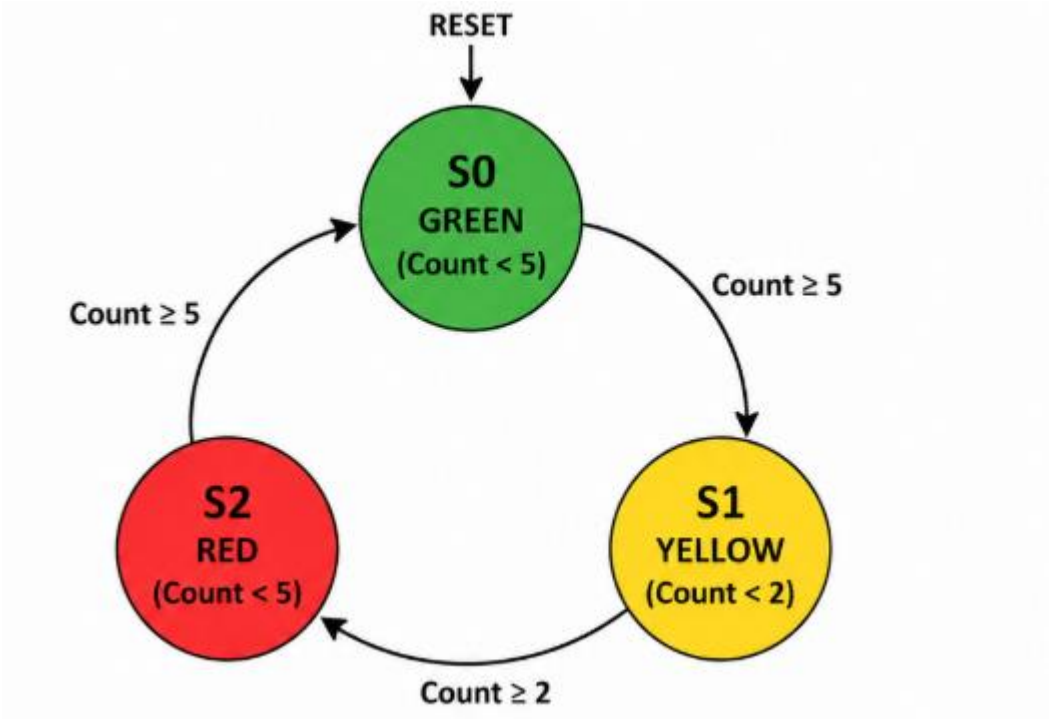
Outputs

- Green LED
- Yellow LED
- Red LED

Software Used

- Verilog HDL
- EDA Playground
- Icarus Verilog Simulator
- EPWave Waveform Viewer

5. State Diagram



6. State Transition Table

Present State	Timer Condition	Next State	Output
GREEN (S0)	Count ≥ 5	YELLOW (S1)	Green = 1
YELLOW (S1)	Count ≥ 2	RED (S2)	Yellow = 1
RED (S2)	Count ≥ 5	GREEN (S0)	Red = 1

7. Verilog Implementation

The controller is implemented using the three-process FSM approach:

Process 1: State Register

- Updates the present state on every positive clock edge.

Process 2: Next-State Logic

- Determines the next state depending on the timer value.

Process 3: Output Logic

- Activates the appropriate traffic light according to the current state.

This modular approach improves code readability, simplifies debugging, and follows good hardware design practices.

8. Codes

Design

```
`timescale 1ns/1ps

module traffic_light(

   input clk,
```

```
    input reset,
    output reg red,
    output reg yellow,
    output reg green
);
parameter GREEN  = 2'b00;
parameter YELLOW = 2'b01;
parameter RED    = 2'b10;
reg [1:0] present_state, next_state;
reg [3:0] count;
always @(posedge clk or posedge reset)
begin
    if(reset)
    begin
        present_state <= GREEN;
        count <= 0;
    end
    else
    begin
        present_state <= next_state;
        if(present_state == next_state)
            count <= count + 1;
        else
            count <= 0;
    end
end
always @(*)
begin
    case(present_state)
        GREEN:
        begin
```

```
            if(count >= 5)
                next_state = YELLOW;
            else
                next_state = GREEN;
        end
        YELLOW:
        begin
            if(count >= 2)
                next_state = RED;
            else
                next_state = YELLOW;
        end
        RED:
        begin
            if(count >= 5)
                next_state = GREEN;
            else
                next_state = RED;
        end
        default:
            next_state = GREEN;
    endcase
end
always @(*)
begin
    red = 0;
    yellow = 0;
    green = 0;
    case(present_state)
        GREEN:
            green = 1;
```

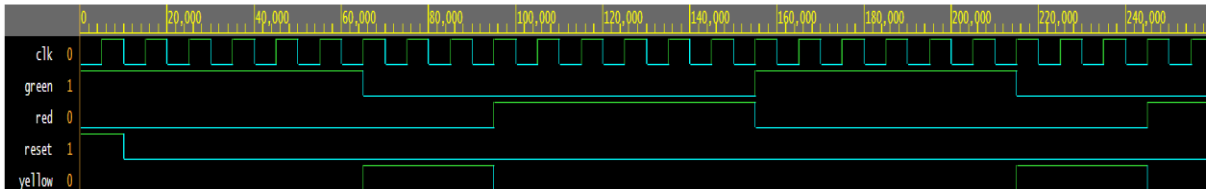
```
        YELLOW:
            yellow = 1;
         RED:
            red = 1;
        default:
        begin
            red = 0;
            yellow = 0;
            green = 0;
        end
    endcase
end
endmodule
```

Testbench

```
`timescale 1ns/1ps
module traffic_light_tb;
reg clk;
reg reset;
wire red;
wire yellow;
wire green;
traffic_light uut (
    .clk(clk),
    .reset(reset),
    .red(red),
    .yellow(yellow),
    .green(green)
);
always #5 clk = ~clk;
```

```
initial
begin
    $dumpfile("dump.vcd");
    $dumpvars(0, traffic_light_tb);
end
initial
begin
    clk = 0;
    reset = 1;
    #10;
    reset = 0;
    #250;
    $finish;
end
initial
begin
    $display("Time\tGreen\tYellow\tRed");
    $monitor("%0t\t%b\t%b\t%b",
            $time, green, yellow, red);
end
endmodule
```

9. Simulation Results



10. Conclusion

The Traffic Light Controller was successfully designed and simulated using Verilog HDL. The project demonstrates the practical application of Finite State Machines and synchronous digital design. By implementing the three-process FSM methodology, the controller achieved correct state transitions and reliable operation. Simulation results verified the functionality of the design, making it a suitable implementation for educational purposes and a strong foundation for advanced traffic management systems.

References

1. M. Morris Mano, *Digital Design*, Pearson Education.
2. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*.
3. EDA Playground Documentation.
4. Icarus Verilog User Guide.
5. Course Notes on Digital Logic Design and Finite State Machines.